

Tema 6: Combinación de algoritmos simétricos y asimétricos

Docente : Héctor Xavier Limón Riaño
email: xavier120@hotmail.com

Introducción

- ▶ En este tema se verá con más detalle como se utiliza típicamente la criptografía asimétrica
- ▶ Siendo responsable de una parte crítica de la privacidad de la comunicación, pero no de toda
- ▶ Usualmente la criptografía asimétrica se usa para establecer una **sesión de confianza** entre dos entidades
- ▶ La comunicación en esa sesión está protegida con métodos simétricos, los cuales son mucho más rápidos

Intercambio de llaves AES con RSA

- ▶ Básicamente es el uso de RSA para cifrar llaves AES que serán compartidas en una comunicación
- ▶ Para establecer la sesión de confianza, ambas partes deben tener el certificado de la otra parte (los cuales incluyen las llaves públicas)
- ▶ Un protocolo simple para transmitir un solo mensaje de emisor a receptor, que incluya llaves simétricas es **agreed-upon transmsion**, en éste se manda un solo flujo de bytes con todos los datos necesarios concatenados
- ▶ Para sesiones más largas con mensajes enviados por ambas partes se verán otros métodos

Intercambio de llaves AES con RSA

- ▶ El flujo de transmisión del protocolo incluye:
 1. La llave AES, IV y llave MAC cifradas con la llave pública del receptor
 2. La firma digital sobre la concatenación (no cifrada) de la llave AES, IV y llave MAC
 3. Los bytes del mensaje transmitido, cifrados con la llave AES e IV
 4. El HMAC sobre toda la transmisión (osea todo lo de arriba) utilizando la llave MAC

Intercambio de llaves AES con RSA

El receptor sigue el siguiente proceso:

- ▶ Separa las partes 1, 2, 3, 4
- ▶ Descifra 1 mediante su llave privada, obteniendo la llave AES, IV y llave MAC
- ▶ Verifica la firma digital (2), mediante el certificado del emisor (verificando también el certificado de ser necesario)
- ▶ Calcula el HMAC de 1, 2, y 3 con la llave MAC y lo compara con 4 para ver si es consistente
- ▶ Descifra 3 mediante la llave AES y el IV

Ejercicio

- ▶ Implementar agreed-upon transmission usando sockets TCP
- ▶ En este caso el cliente envía el mensaje de una sola vía al servidor, el servidor (receptor) comprueba que todo se haya recibido bien e imprime el mensaje recibido
- ▶ El maestro implementará la parte del cliente
- ▶ Tu trabajo consiste en hacer la parte del servidor (se te dará una plantilla)

Llaves de sesión

- ▶ Una llave de sesión es básicamente la llave AES utilizada en el ejemplo anterior
- ▶ Esta llave debe ser usada sólo para una sesión y descartada permanentemente, por esto también se les suele llamar **efímeras**
- ▶ También se les llama "bulk encryption keys" o llaves para el cifrado en masa, ya que los datos de la sesión son cifrados con esta llave

Llaves de sesión

- ▶ Las llaves simétricas son buenas llaves de sesión por varios motivos:
 - ▶ Son fáciles de crear (bytes random), no requieren parámetros especiales
 - ▶ Se pueden derivar de un secreto base usando funciones de derivación de llaves (se verán más adelante)
 - ▶ Los algoritmos de cifrado simétrico son rápidos, AES es típicamente cientos de veces más rápido que RSA
 - ▶ No son llaves que debas mantener mucho tiempo pues son compartidas, por lo que son desechables. En contraste, las llaves asimétricas son útiles para identificación a largo plazo, gracias a que no se comparte la llave privada

Criptografía simétrica y asimétrica

- ▶ Las técnicas asimétricas se complementan con las simétricas
- ▶ Usando certificados, las llaves asimétricas pueden ser usadas como forma de probar la identidad
- ▶ Una vez esto sucede, las llaves de sesión con un tiempo de vida corto hacen el trabajo real de transmisión entre entidades autenticadas

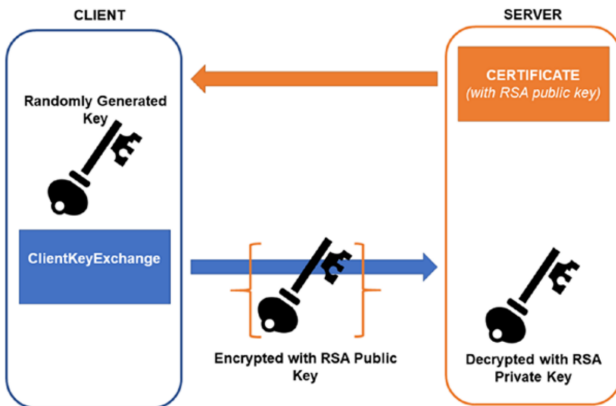
Diffie-Hellman y acuerdo de llaves

Introducción

- ▶ Es un método de criptografía asimétrica
- ▶ También llamado DH
- ▶ Una variante de este método que utiliza curvas elípticas se conoce como ECDH (Eliptic-Curve Diffie-Hellman)
- ▶ DH se utiliza sólo para intercambio de llaves, por esto también se le llama intercambio de llaves Diffie-Hellman

Introducción

- ▶ A parte de firmas digitales, RSA se usa también para transportar llaves, como se vio en agreed-upon-transmission
- ▶ La figura muestra el transporte de llaves en el handshake de TLS 1.2



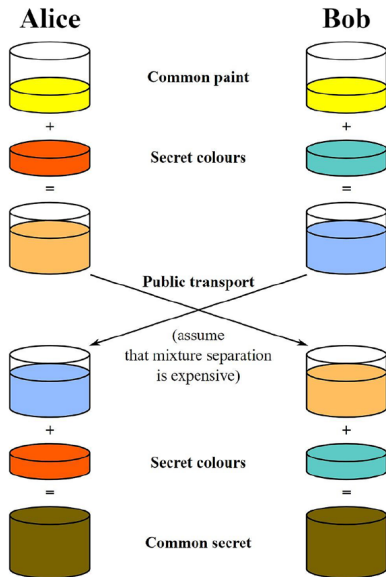
Llaves DH

- ▶ En contraste con RSA, DH crea llaves aparentemente de la nada
- ▶ No se necesita compartir secretos entre partes
- ▶ Lo que hace DH es intercambiar parámetros públicos que le permiten a las partes computar simultáneamente la misma llave en ambos lados
- ▶ A este proceso se le conoce como **Acuerdo de llaves** (key agreement)

Proceso DH

- ▶ Lo primero que hace DH es crear, para cada participante, un par de números matemáticos, uno privado y otro público
- ▶ El protocolo de acuerdo entre llaves DH y ECDH requiere que tanto el emisor como el receptor tengan su par de llaves (sus números)
- ▶ Las dos partes intercambian sus números públicos
- ▶ Con el número público externo y el número privado local se genera la misma llave compartida en ambos lados

Proceso DH

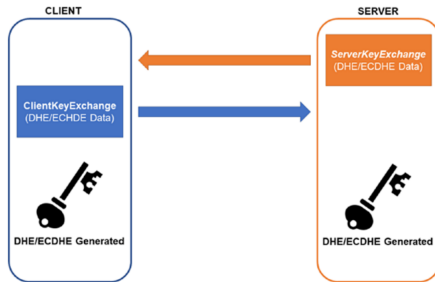


Proceso DH

- ▶ Notar que este proceso no permite el envío de datos arbitrarios, como en el intercambio de llaves RSA
- ▶ Todo el proceso sirve sólo para establecer un acuerdo con respecto a un dato aleatorio
- ▶ Ninguna de las partes escoge el contenido del mensaje
- ▶ El contenido aleatorio puede ser y usualmente es una llave simétrica o bien un dato que permite derivar una llave simétrica

Proceso DH

- ▶ También notar que a diferencia de el transporte de llaves con RSA, **DH requiere de comunicación bidireccional, lo cual puede agregar problemas de seguridad**
- ▶ En la realidad lo anterior puede no ser un problema tan grave, considerando que en internet todos son libres de comunicarse con todos
- ▶ **TLS 1.2 también soporta acuerdo de llaves DH, esto es muestra en la figura**



DH en cryptography

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives import hashes
3 from cryptography.hazmat.primitives.asymmetric import dh
4 from cryptography.hazmat.primitives.kdf.hkdf import HKDF
5
6 # Parametros reusables
7 # Deben ser iguales en ambos lados
8 parameters = dh.generate_parameters(generator=2, key_size=2048,
9                                     backend=default_backend())
10 private_key = parameters.generate_private_key()
11
12 # Esta es la que se tiene que intercambiar
13 public_key = private_key.public_key()
```

DH en cryptography

```
1  # En algun punto se obtiene otra_public_key
2  shared_key = private_key.exchange(otra_public_key)
3
4  # shared_key se podria usar directamente
5  # pero es mejor combinar con algoritmo de derivacion de llave
6  derived_key = HKDF(algorithm=hashes.SHA256(),
7                      length=32,
8                      salt=None,
9                      info=b'handshake data',
10                     backend=default_backend()).derive(shared_key)
```

Acuerdo de llaves DH

- ▶ A diferencia de el transporte de llaves RSA, **hay menos parámetros a configurar**
- ▶ Los parámetros de DH **deben ser iguales de los dos lados**
- ▶ Sólo hay dos parámetros básicamente:
 - ▶ **Tamaño de la llave: debe ser de al menos 2048 bits, dado que una llave más pequeña es propensa a ser factorizada**
 - ▶ **Generador: sólo hay dos posibles valores, 2 y 5.** Desde el punto de vista de seguridad no importa que valor se toma

Acuerdo de llaves DH

- ▶ Es importante no estar recalculando los parámetros para cada llave que se necesite
- ▶ La generación de parámetros no es tan rápida, pero la generación de llaves a partir de los parámetros si lo es
- ▶ No es un problema de seguridad el reuso de los parámetros para distintas llaves
- ▶ También de forma alternativa puede usarse ECDH que es más rápido

Acuerdo de llaves DH

- ▶ La otra recomendación es **usar una función de derivación de llaves (HKDF en el ejemplo)**, esto también ayuda a reducir el tamaño de la llave para ser usada por AES
- ▶ A diferencia del transporte de llaves RSA, **DH no tiene autenticación integrada**
- ▶ **Una o ambas partes deben firmar los datos públicos intercambiados**

ECDH

- ▶ Es una variante de DH que se ha vuelto popular en tiempos recientes
- ▶ Funciona de la misma forma que DH, pero usa curvas elípticas para algunos de los cálculos matemáticos internos
- ▶ En cryptography se usa casi de la misma forma que DH

ECDH en cryptography

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives import hashes
3 from cryptography.hazmat.primitives.asymmetric import ec
4 from cryptography.hazmat.primitives.kdf.hkdf import HKDF
5
6 # solo hay que elegir un tipo de curva
7 private_key = ec.generate_private_key(ec.SECP384R1(),
8                                     default_backend())
9
10 # Esta es la llave a intercambiar
11 public_key = private_key.public_key()
```

ECDH en cryptography

```
1  # Otra_publica es la de la otra parte
2  shared_key = private_key.exchange(ec.ECDH(), otra_publica)
3
4  #Derivar llave
5  derived_key = HKDF(algorithm=hashes.SHA256(),
6                      length=32,
7                      salt=None,
8                      info=b'handshake data ',
9                      backend=default_backend()
10                     ).derive(shared_key)
```

Confidencialidad hacia adelante Diffie-Hellman

Introducción

- ▶ DH y ECDH son preferibles al transporte de llaves RSA
- ▶ Hay varias razones pero la principal es que DH soporta Confidencialidad hacia adelante
- ▶ Esta propiedad significa que incluso si una llave es eventualmente comprometida, no revela nada acerca de comunicaciones previas

Introducción

- ▶ Volviendo al transporte de llaves RSA (las llaves AES se cifran con la llave pública del receptor)
- ▶ Un atacante puede guardar todos los datos capturados de una transmisión, incluyendo el transporte de llaves
- ▶ Aunque en el momento no pueda determinar el contenido, porque no posee la llave privada que necesita, en el futuro podría obtenerla, recordando que las llaves privadas pueden tener un tiempo de vida largo, revelando la información de toda la transmisión
- ▶ Lo que es peor, podría revelar la información de todas las transmisiones capturadas que ocupen esa llave privada

Llaves efímeras

- ▶ DH y ECDH resuelven el problema anterior mediante el uso de **llaves efímeras**
- ▶ RSA sólo produce llaves simétricas efímeras, **pero DH y ECDH producen también llaves asimétricas efímeras**
- ▶ **La idea es que cada acuerdo de intercambio de llaves tenga sus propio par de llaves, que serán desechadas al terminar el intercambio,** así las llaves no estarán almacenadas en ningún lado, por lo que no se podrán robar
- ▶ Cuando se usan llaves efímeras (y siempre deben usarse) se le suele agregar una E (ephimeral) a DH, obteniendo DHE y ECDHE

Llaves efímeras

- ▶ Aun de poder robar un atacante el par de llaves, estás sólo servirían para una sesión pues cada vez se genera un par diferente
- ▶ Se debe tener cuidado en algunas herramientas criptográficas, a veces DH implícitamente significa DHE pero no siempre es el caso, es necesario leer la documentación

Autenticación en DH y ECDH

Introducción

- ▶ Como ya se mencionó DH y ECDH no tienen autenticación integrada, como si lo tiene el transporte de llaves RSA
- ▶ Como las llaves son efímeras, no hay formas de ligarlas a una identidad
- ▶ Recuerda que además de tener confidencialidad se necesita saber con quien te comunicas de forma confidencial (autenticación)

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

Ejercicio

- ▶ Hacer un intercambio de llaves ECDHE mediante sockets
- ▶ Autenticar el intercambio

Protocolos reto-respuesta

- ▶ En el tema de certificados se revisó brevemente este protocolo
- ▶ Este protocolo se trata de que una de **las partes le pruebe a la otra que tiene actualmente control de un secreto compartido o de una llave privada**

Protocolos reto-respuesta

- ▶ Por ejemplo, para un secreto compartido, donde A y B comparten una llave K , osea $K_{A,B}$
- ▶ A puede pedirle a B que cifre un nonce N el cual puede no estar cifrado
- ▶ Por seguridad, es una buena idea que la respuesta de B incluya las identidades de A y B
- ▶ B responderá con $\{A, B, N\}_{K_{A,B}}$
- ▶ Si sólo A y B conocen K entonces aunque alguien intercepte N no podrá cifrarlo

Protocolos reto-respuesta

- ▶ Para el caso asimétrico es más o menos lo mismo pero usando firmas digitales generadas con llaves privadas
- ▶ Si B se comunica con A y quiere asegurarse de que realmente es A
- ▶ B le manda a A un nonce N y le pide que lo firme con su llave privada
- ▶ A responde con un mensaje (para firma RSA) que incluya la identidad de ambos, osea $\{H(B, AmB)\}_{K_A^{-1}}$
- ▶ B verifica con la llave pública de A la firma (usando posiblemente el certificado de A)

Protocolos reto-respuesta

- ▶ Estos algoritmos son simples, pero se pueden implementar mal de muchas formas
- ▶ El nonce debe ser suficientemente largo y aleatorio para que no sea predecible, incluso con conocimiento de transmisiones anteriores
- ▶ En los primeros tiempos de las llaves electrónicas para autos el transmisor usaba un nonce de 16 bits
- ▶ Los ladrones sólo tenían que capturar una transmisión y luego interrogar al sistema hasta que se ciclaran los nonce

Protocolos reto-respuesta

- ▶ Estos protocolos también son propensos a ataques de MITM
- ▶ Si un atacante E quiere convencer a A de que es B
- ▶ Entonces E puede esperar a que B quiera hablar con A , interceptando toda su comunicación
- ▶ Luego E inicia comunicación con A , pretendiendo ser B
- ▶ A manda un reto N para saber que habla con B
- ▶ E le manda N a B
- ▶ B firma N y E lo intercepta, mandándoselo a A
- ▶ A se está comunicando felizmente con E pensando que es B

Protocolos reto-respuesta

- ▶ Una forma de mitigar los problemas de MITM es transmitiendo información que sólo la otra parte verdadera puede usar
- ▶ Por ejemplo, incluso si E logra responder al reto, haciéndose pasar por B , esto no le ayuda si la respuesta de A es mandar a B una llave de sesión cifrada con la llave pública de B , E no podrá descifrar esta información

Protocolos reto-respuesta

- ▶ Alternativamente, A y B pueden usar ECDH con firmas para generar la llave de sesión (como hiciste en el ejercicio anterior)
- ▶ Aunque E pueda interceptar toda la transmisión no podrá tener la llave de sesión (nunca ve la parte privada) y por lo tanto no podrá descifrar ni alterar nada de lo que se transmite
- ▶ A lo mucho lo que puede hacer E es generar un DOS al bloquear las transmisiones

Protocolos reto-respuesta

- ▶ Una vez las identidades se han establecido, **todas las comunicaciones de la sesión deben estar ligadas a esa autenticación**
- ▶ Por ejemplo, al establecer la identidad se debió compartir una llave MAC, que debe ser usada para generar un MAC en todas las transmisiones

Protocolos reto-respuesta

- ▶ A veces, algunos datos de inicialización deben ser transmitidos antes de poder establecer la comunicación cifrada
- ▶ Toda esta información debe ser vinculada a la autenticación en algún momento
- ▶ Por ejemplo, después de establecer las llaves de sesión, y tener un canal seguro, se podría mandar el hash de todos los datos que no se habían autenticado al inicio
- ▶ Si no coinciden entre ambas partes, se podría asumir que hubo un ataque

Problemas comunes de seguridad

Introducción

- ▶ Crear protocolos criptográficos seguros es muy complejo
- ▶ Aunque creas saber lo que estás haciendo, no intentes crear tu propio protocolo
- ▶ La historia de la criptografía está plagada de ejemplos donde un protocolo que se pensó era seguro fue vulnerado, desarrollados por criptografos muy capaces

Introducción

- ▶ Por ejemplo, en todo lo que se ha visto en este tema, no se utilizó algo para mitigar ataques de repetición
- ▶ ¿Te diste cuenta de eso?
- ▶ Otro ejemplo, sólo se generó una llave de sesión, cuando tendría que haber dos, una para el emisor y otra para el receptor, para una comunicación full-duplex, ¿Cuál es el problema con esto?
- ▶ Recuerda que nunca hay que repetir la misma llave e IV y el IV aunque no necesita ser privado tendría que ser suficientemente aleatorio

Introducción

- ▶ Es muy difícil tomar en cuenta todas las posibles situaciones de ataque y recordar todos los detalles o incluso conocerlos
- ▶ Esto es verdadero incluso para los expertos
- ▶ En general, **no diseñes tus propios protocolos**
- ▶ Utiliza protocolos existentes tanto como puedas, así como implementaciones existentes de poderse
- ▶ **Otra vez recuerda: YANAC**

Ransomware

- ▶ Así como la criptografía puede ser utilizada para defenderse de los atacantes, **la criptografía también puede ser usada para atacar**
- ▶ Los atacantes pueden hacer mucho dinero haciendo el mal, por lo que pueden estar muy motivados para crear métodos creativos y eficientes
- ▶ Uno de esos métodos creadas por atacantes es el Ransomware

Ransomware

- ▶ Es básicamente software que cifra tus archivos y se rehúsa liberarlos hasta que le pagues al extorsionador que está detrás de todo
- ▶ La criptografía que se usaba al inicio en estos métodos era simplista e ingenua
- ▶ El Ransomware cifraba cada archivo con una llave AES aleatoria diferente, pero todas las llaves AES eran almacenadas en el sistema en un solo archivo, el cual no estaba cifrado
- ▶ Esto se hacía para que fuera fácil descifrar, pero un analizador forense también podía encontrar el archivo

Ransomware

- ▶ Los atacantes cambiaron a métodos criptográficos asimétricos como solución
- ▶ Una llave pública puede ser alojada en el sistema de la víctima y la llave privada ser resguardada por el atacante
- ▶ La idea sería cifrar todo con la llave pública, para que sólo se pueda descifrar con la llave privada, pero...
- ▶ No hay cifradores asimétricos eficientes, o no tienen la capacidad de cifrar archivos grandes (como RSA)
- ▶ La víctima se daría cuenta del ataque mucho antes de que terminara

Ransomware

- ▶ Lo que se puede hacer es cifrar los archivos con llaves AES diferentes y después cifrar la llave AES con la llave pública
- ▶ Como las llaves son pequeñas, 16 bytes para AES-128 y 32 bytes para AES-256, no sería ineficiente
- ▶ Ya que se cifraron se pueden almacenar en el sistema de la víctima

Ransomware

- ▶ Hay dos variaciones ingenuas al ataque anterior
- ▶ En la primera el atacante genera su par de llaves antes de cualquier ataque
- ▶ El atacante hard-codea la llave pública en el propio malware
- ▶ Ya que el ataque surgió efecto, la víctima paga el rescate y se le manda la llave privada
- ▶ La debilidad obvia es que si el mismo malware se usa para infectar múltiples sistemas, entonces la misma llave privada por la que pagó alguien le servirá a los demás

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

Ransomware

- ▶ El ransomware moderno soluciona todos los problemas anteriores con una aproximación muy ingeniosa
- ▶ Primero el atacante genera un par de llaves asimétricas de larga duración, llamémoslas llaves permanentes
- ▶ Luego el atacante crea el ransomware y hard-codea la llave pública permanente
- ▶ Cuando el malware se activa se genera un nuevo par de llaves en la víctima, llamémoslas llaves locales
- ▶ Inmediatamente cifra la llave privada local con la llave pública permanente y borra la llave local privada plana

Ransomware

- ▶ Entonces el malware empieza a cifrar los archivos del disco usando AES-CTR o AES-CBC
- ▶ Cada archivo se cifra con una llave AES diferente y luego esa llave es cifrada con la llave local pública y destruida después de cifrar el archivo
- ▶ Las llaves AES cifradas pueden ser descifradas con la llave local privada
- ▶ Ahora el atacante contacta a la víctima (o vise-versa) y demanda que se pague el rescate

Ransomware

- ▶ Si la víctima paga el rescate (usualmente con criptomonedas como Bitcoin), el atacante le manda una información de autenticación al malware
- ▶ El malware le transmite al atacante la llave privada local cifrada
- ▶ Usando la llave privada permanente, el atacante descifra la llave privada local y se la manda a la víctima
- ▶ Ahora todas las llaves AES pueden ser descifradas junto con los archivos
- ▶ La llave privada permanente nunca es revelada